



PIMPRI CHINCHWAD UNIVERSITY
School of Engineering & Technology



Academic Year: 2026-2027

Sem: I

Course: Cloud Computing Lab

Course code: UBTFY113/SEC

Name: Kartik Pravin Ingle

Remark:-

Roll No: 20

Signature :-

Practical 1: Horizontal and Vertical Scaling using Python Multithreading

Aim:

To implement and compare Horizontal Scaling and Vertical Scaling using Python multithreading by measuring their execution time, and to understand how scaling techniques improve resource utilization and performance in Cloud Computing.

Theory:

Introduction

Cloud Computing enables applications to handle varying workloads by providing scalable computing resources. As the number of users or requests increases, cloud platforms must ensure that applications continue to perform efficiently without service interruption. This is achieved through scaling, which allows computing resources to be increased or decreased according to demand.

Major cloud service providers such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP) provide scaling features that automatically allocate resources based on workload, ensuring high availability and better performance.

Scaling techniques are mainly classified into:

1. Horizontal Scaling (Scaling Out/In)
2. Vertical Scaling (Scaling Up/Down)

Horizontal Scaling (Scaling Out)

Horizontal Scaling refers to increasing the number of servers or virtual machines to distribute the workload among multiple systems. Instead of upgrading a single server, additional servers are added to share the processing tasks.

For example, when an online shopping website receives heavy traffic during a sale, multiple server instances are created to process user requests simultaneously.

Characteristics

- Adds multiple servers or virtual machines.
- Distributes workload across several systems.
- Improves availability and fault tolerance.
- Easily expands resources according to demand.

Advantages

- High reliability and availability.
- Efficient load distribution.
- Suitable for large-scale cloud applications.
- Supports a large number of concurrent users.
- Easy to increase or decrease resources.

Disadvantages

- More complex infrastructure.

- Requires load balancing mechanisms.
- Data synchronization may be required.

Applications

- Google Search
 - Amazon
 - Netflix
 - Facebook
 - YouTube
-

Vertical Scaling (Scaling Up)

Vertical Scaling refers to increasing the hardware capacity of a single server by upgrading CPU, RAM, storage, or processing power instead of adding more servers.

For example, upgrading a virtual machine from 2 CPU cores and 4 GB RAM to 8 CPU cores and 32 GB RAM.

Characteristics

- Enhances the performance of one machine.
- Simple to implement.
- No load balancing required.
- Limited by maximum hardware capacity.

Advantages

- Easy implementation.
- No major changes in application architecture.
- Suitable for applications that cannot be distributed.

Disadvantages

- Hardware upgrade limitations.
- Single point of failure.

- Downtime may occur during upgrades.
- Higher cost for large upgrades.

Applications

- Database servers
 - Virtual machine upgrade
 - AWS EC2 instance resizing
 - Enterprise applications
-

Python Multithreading

Multithreading allows multiple tasks to execute concurrently within a single program. In Python, the threading module is used to create and manage threads, enabling parallel execution of independent tasks.

Important Methods Used

- `Thread()` – Creates a new thread.
- `start()` – Starts thread execution.
- `join()` – Waits until the thread completes execution.

In this practical:

- Horizontal Scaling is simulated by creating multiple threads that execute workloads simultaneously, representing multiple cloud servers.
- Vertical Scaling is simulated by using a single thread with a larger workload, representing a more powerful server.

Program Explanation

Horizontal Scaling

- Multiple threads are created.
- Each thread performs the same workload independently.
- Threads execute simultaneously.
- Total execution time is measured.

- Simulates multiple cloud instances processing tasks in parallel.

Vertical Scaling

- A single thread performs a larger workload.
- Increased computational power represents upgraded hardware.
- Execution time is measured.
- Simulates increasing the capacity of one server instead of adding multiple servers.

Applications in Cloud Computing

Scaling is an essential feature of cloud computing because it enables systems to efficiently manage changing workloads. It helps organizations to:

- Handle increasing user requests.
- Improve application performance.
- Optimize resource utilization.
- Reduce operational costs.
- Maintain continuous service availability.
- Prevent system overload during peak traffic.

Common applications include:

- E-commerce websites during seasonal sales.
- Banking and financial systems.
- Online gaming platforms.
- Video streaming services.
- Cloud-hosted enterprise applications.

Algorithm

1. Import the threading and time modules.
2. Create a function to simulate CPU-intensive work.
3. Create multiple threads for Horizontal Scaling.
4. Start all threads simultaneously.

5. Wait for all threads to complete using join().
6. Measure and record the execution time.
7. Create a single thread with a larger workload for Vertical Scaling.
8. Measure its execution time.
9. Compare the execution times of both scaling techniques.
10. Display the results.

Output

```

Jupyter horizontal byverticalsimulationbyk Last Checkpoint: 20 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

if __name__ == '__main__':
    print("Resource scaling simulation(horizontal vs vertical)\n")

    start=time.time()
    horizontal_scaling(num_threads=4)
    h_time=time.time()-start

    start=time.time()
    vertical_scaling(power=4)
    v_time=time.time()-start

    print(f" Time taken for horizontal scaling: {h_time :.2f} seconds")
    print(f" Time taken for vertical scaling: {h_time :.2f} seconds")

Resource scaling simulation(horizontal vs vertical)

.....Horizontal scaling simulation.....
[Horizontal]Task 0 starting with power 1
[Horizontal]Task 1 starting with power 1
[Horizontal]Task (task_id) completed
[Horizontal]Task (task_id) completed
[Horizontal]Task 2 starting with power 1
[Horizontal]Task 3 starting with power 1
[Horizontal]Task (task_id) completed
[Horizontal]Task (task_id) completed
Horizontal scaling complete.....

----- Vertical scaling Simulation-----
[Vertical]Task 0 starting with power 4
[Vertical]Task (task_id) completed
Vertical scaling complete.

Time taken for horizontal scaling: 0.15 sconds
Time taken for vertical scaling: 0.15 sconds

```

```

Jupyter horizontal byverticalsimulationbyk Last Checkpoint: 20 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

if __name__ == '__main__':
    print("Resource scaling simulation(horizontal vs vertical)\n")

    start=time.time()
    horizontal_scaling(num_threads=4)
    h_time=time.time()-start

    start=time.time()
    vertical_scaling(power=4)
    v_time=time.time()-start

    print(f" Time taken for horizontal scaling: {h_time :.2f} seconds")
    print(f" Time taken for vertical scaling: {h_time :.2f} seconds")

Resource scaling simulation(horizontal vs vertical)

.....Horizontal scaling simulation.....
[Horizontal]Task 0 starting with power 1
[Horizontal]Task 1 starting with power 1
[Horizontal]Task (task_id) completed
[Horizontal]Task (task_id) completed
[Horizontal]Task 2 starting with power 1
[Horizontal]Task 3 starting with power 1
[Horizontal]Task (task_id) completed
[Horizontal]Task (task_id) completed
Horizontal scaling complete.....

----- Vertical scaling Simulation-----
[Vertical]Task 0 starting with power 4
[Vertical]Task (task_id) completed
Vertical scaling complete.

Time taken for horizontal scaling: 0.15 sconds
Time taken for vertical scaling: 0.15 sconds

```

The program successfully simulates Horizontal Scaling and Vertical Scaling using Python multithreading. It displays the execution time for both techniques, allowing comparison of their performance and demonstrating how different scaling methods improve resource utilization in cloud computing environments

Conclusion

The practical successfully demonstrated the concepts of Horizontal Scaling and Vertical Scaling using Python multithreading. Horizontal Scaling distributes work among multiple threads, similar to adding multiple cloud servers, while Vertical Scaling increases the workload of a single thread, representing a more powerful server. By comparing their execution times, the experiment illustrates how different scaling techniques can improve performance, optimize resource utilization, and support efficient application deployment in cloud computing environments.